

TetraTHON 2026 — AgriTech

Precision Crop Advisory System & Post-Harvest Loss Reduction Planner

Simple Phase-Wise Build Plan — 4 People, 4 Chunks, One Person at a Time

Every phase is split into exactly 4 chunks. Each chunk lists exactly what to build.

Prototype (Week 1) → Hackathon MVP (Week 2) → Pilot → Scale

Version 1.0 | July 17, 2026

Table of Contents

1. How This Plan Works (Read This First)

The whole project — from the very first prototype to a fully scaled product — is broken into phases. Each phase is broken into exactly 4 chunks. Each chunk is one person's full responsibility, start to finish.

1.1 The One Rule That Matters Most

Only one person builds at a time. Nobody works on frontend while someone else works on backend. Instead, each person builds one complete feature — its screen, its logic, and its backend — from start to finish, and hands over a working app to the next person.

- Person 1 builds their chunk completely, and the app works and is live.
- Person 2 then builds their chunk on top of it — the app still works, plus the new feature.
- Person 3 then builds their chunk on top of that. And so on.
- This repeats for every phase: P1 → P2 → P3 → P4, every time.

This means: no merge conflicts, no confusion about who is doing what, and at every moment the app is either working or one person is actively fixing it — never half-broken from two people editing the same thing.

1.2 Simple Handoff Rule

Before you hand off to the next person, do these 3 things:

1. Make sure the whole app still works — not just your new part, everything built before you too.
2. Push your code live (deploy it) so the next person can see and use it immediately.
3. Write 3–4 lines telling the next person: what you built, how to test it, and where they should start.

1.3 How to Read Each Chunk Below

Every chunk below tells you exactly what to build — as a numbered checklist. Follow it top to bottom. When you finish every step, check the green "Done when" line — if that's true, you're finished and the next person can start.

2. Phase 0 — Prototype (Week 1, Days 1–7)

Goal of this phase: build a working, simple version of both modules using fake/mock data, so it can be demoed and submitted for pre-screening. Nothing here needs to be perfect — it needs to work, end to end, for all 4 crops and 5 locations.

Chunk 1 — Project Foundation & Rule Tables

Owner: P1 | Timing: Day 1 (1 day)

You are building the skeleton that everyone else builds on top of. Nothing fancy — just a working, empty app that is live on the internet, plus the base data everyone else needs.

What to build, step by step:

1. Create a GitHub repository with two folders: /frontend and /backend.
2. Set up the backend using FastAPI (Python) with a single working endpoint: GET /health that returns { "status": "OK" }.
3. Set up the frontend using React (Vite) + Tailwind CSS, with a homepage that calls the /health endpoint and shows the result.
4. Create the shared data model: a Location table (5 rows — pick 5 real locations), a Crop table (4 rows — pick 4 real crops), and a FarmerSession table (empty for now, just the structure).
5. Load this seed data into SQLite so both frontend and backend can read it.
6. Research and write the rule tables for irrigation, fertiliser, and pest windows for the 4 chosen crops. Save as a simple CSV or JSON file — this is what Chunk 2 will use.
7. Write down the two user flows in plain text or a simple diagram: (a) the Advisory flow, (b) the Post-Harvest flow — this becomes the architecture diagram later.
8. Deploy the frontend to Vercel and the backend to Render or Railway. Confirm both are reachable from a public link.
9. Push everything to GitHub with a short README explaining how to run the project locally.

Done when: The app is live at a public link, the health-check page loads and shows OK, and the seed data (5 locations, 4 crops) is visible from both frontend and backend.

Chunk 2 — Module A — Crop Advisory Engine

Owner: P2 | Timing: Day 2–3 (2 days)

You are building the first real feature: a farmer fills a form and gets 3 ranked advisories. Everything runs on fake weather data for now — that becomes real in Week 2.

What to build, step by step:

1. Build the intake form (frontend): a dropdown for location, a dropdown for crop, a date picker for sowing date, a text/select field for a recent weather observation, and an optional photo upload button (the button can exist and store the file, but you don't need to use the photo yet — that comes later).
2. Build a mocked weather adapter: for each of the 5 seeded locations, write a fake 7-day forecast (temperature, rainfall chance) as a JSON file the backend reads from.

3. Build the backend endpoint POST /advisory that takes crop, location, sowing date, and weather observation as input.
4. Build the rule-based ranking engine: using Chunk 1's rule tables, write logic that checks the crop's current growth stage (from sowing date) and the mocked weather, and picks the 3 most relevant advisories (irrigation, fertiliser, pest/disease).
5. Add a confidence score to each advisory: High / Medium / Low, based on simple logic (e.g. High if the rule matched exactly, Medium if it's a fallback rule, Low if data was incomplete).
6. Write a plain-language sentence template for each advisory type, e.g. "Water your crop today — soil moisture is low and no rain is expected in the next 3 days." Fill in real numbers/details from the request.
7. Test this for every combination of the 4 crops and 5 locations (20 combinations total) — make sure none of them crash and the text reads sensibly.
8. Deploy your changes on top of Chunk 1's live app. Confirm Chunk 1's health-check page still works.

Done when: A farmer can fill in the form, hit submit, and see 3 ranked advisories with a confidence tag and a plain-language explanation for each — working correctly for all 4 crops × 5 locations, live on the deployed link.

Chunk 3 — Module B — Post-Harvest Loss Planner

Owner: P3 | Timing: Day 4–5 (2 days)

You are building the second feature: a farmer enters their harvest details and gets a Sell / Store / Transport recommendation. This must not break anything Chunk 2 built.

What to build, step by step:

1. Build the intake form (frontend): a dropdown for crop, a number input for quantity, a dropdown for storage condition (open / warehouse / cold storage), and a dropdown for location.
2. Create synthetic mandi (market) price data: a CSV with daily prices for each crop across 5 markets — make the numbers realistic enough to show a believable trend.
3. Build the backend endpoint POST /post-harvest that takes crop, quantity, storage condition, and location as input.
4. Build the spoilage model: a simple function that estimates how much value is lost per day, based on the crop and storage condition (e.g. produce in open storage loses value faster than cold storage).
5. Build the transport cost model: a simple cost-per-kilometre function using the straight-line distance between the farmer's location and each of the 5 markets.
6. Build the decision engine: calculate the expected net return for 3 options — Sell now, Store for N days, or Transport to the best market — and return whichever option gives the highest expected return, with the number shown.
7. Test this for a range of crop / quantity / storage / location combinations to make sure the recommendation logic makes sense and doesn't crash.
8. Deploy your changes on top of Chunk 2's live app. Re-test the advisory form from Chunk 2 to confirm it still works.

Done when: A farmer can enter crop, quantity, storage condition, and location, and get a Sell / Store / Transport recommendation with an expected-return number — live on the deployed link, without breaking Module A.

Chunk 4 — Integration, Polish & Submission

Owner: P4 | **Timing:** Day 6–7 (2 days)

You are connecting both modules into one clean demo and getting everything ready to submit. This chunk is split into two halves: build (Day 6) and submission prep (Day 7).

What to build, step by step:

1. Day 6 — Build a single dashboard screen where you pick one farmer/crop scenario and see both Module A's advisories and Module B's recommendation together, side by side.
2. Day 6 — Add two simple line charts to the dashboard: one showing the spoilage curve over time, one showing the price trend — use Recharts or Chart.js.
3. Day 6 — Do a UI/UX pass: consistent colours and spacing, use icons instead of dense text where possible, and check the layout works on a phone screen.
4. Day 6 — Write a clear README: what the project does, how to run it, and 2–3 screenshots. Draw a simple architecture diagram (boxes: Frontend → Backend → Rule Engine → Data). Record a short demo GIF or video.
5. Day 7 (morning) — Build the submission PPT: Title & team, Problem Understanding, Approach (architecture + tech stack + data flow), Demo screenshots, Market Fit & Real-World Relevance, Roadmap (Phase 0 → 1 → 2), Closing slide with links.
6. Day 7 (morning) — Rehearse the demo out loud at least twice. Re-test all 4×5 crop/location combinations one final time.
7. Day 7 (morning) — Ask someone outside the team to try the demo cold. Fix anything that confuses them.
8. Day 7 (afternoon) — Submit the PPT and the GitHub link. Keep this time as buffer — don't plan new work into it.

Done when: One connected dashboard shows both modules working together for a chosen scenario, the repo has a clear README and visible commit history, the PPT is ready, and the submission has gone in.

3. Phase 1 — Hackathon MVP (Week 2, Days 8–14)

This phase starts only once your team is selected and receives the full problem statement at the July 31 inauguration. Goal of this phase: replace the fake/mockered parts with real ones, and add the stretch features that were skipped in Week 1. The same rotation continues — Person 1 leads again.

If the organisers announce a much shorter on-site window (for example 24–48 hours) instead of a full week, shrink each chunk below to roughly a quarter of the days shown, in the same order, and drop the lowest-priority steps first (leaf classifier and SMS alert are the first to trim).

Chunk 5 — Real Data — Weather & Market Prices

Owner: P1 | Timing: Day 8–9 (2 days)

You are replacing the fake weather and price data from Week 1 with real, live data — but keeping the fake data as a safety net in case the internet or an API fails during the demo.

What to build, step by step:

1. Read the full hackathon problem statement carefully and note anything that changes what you need to build compared to the Week 1 prototype.
2. Sign up for a free OpenWeatherMap (or IMD) API key.
3. Build a real weather adapter that calls the live API for the 5 seeded locations and returns data in the exact same format the mocked adapter used, so nothing else in the app needs to change.
4. Add a fallback: if the live weather call fails or times out, automatically use Week 1's mocked data instead, silently — the user should never see an error screen.
5. Do the same for market prices: connect to Agmarknet or data.gov.in (or download and refresh their CSV) for the crops/markets you're tracking, with the same silent fallback to the synthetic data if it fails.
6. Test the fallback on purpose — turn off your internet or use a fake API key — and confirm the app keeps working using the mocked data.
7. Deploy your changes on top of Chunk 4's live app. Confirm nothing from Week 1 broke.

Done when: The app runs on real weather and price data end-to-end. If the network drops or an API fails, the app quietly falls back to the mocked data instead of crashing or showing an error.

Chunk 6 — Leaf-Disease Photo Classifier

Owner: P2 | Timing: Day 10–11 (2 days)

You are making the leaf-photo upload button (built in Chunk 2 but unused) actually work — a real, if simple, AI model that looks at a leaf photo and predicts a disease or pest.

What to build, step by step:

1. Download a subset of the PlantVillage dataset (Kaggle) for 1–2 of your 4 crops — pick the ones with the most/clearest sample images.
2. Fine-tune a MobileNetV2 model (transfer learning, not training from scratch) on this subset using PyTorch or TensorFlow.
3. Export the trained model to TFLite format so it runs fast and light.

4. Build a backend endpoint POST /leaf-classify that accepts an uploaded photo, runs it through the model, and returns a predicted disease/pest name with a confidence percentage.
5. Wire the existing photo-upload field on the intake form to call this endpoint and display the result alongside the other 3 advisories.
6. Clearly label the result as "AI-assisted, not a certified diagnosis" so it's not mistaken for professional agronomic advice.
7. Test with 8–10 sample leaf photos (mix of healthy and diseased) to check the predictions look reasonable.
8. Deploy on top of Chunk 5's live app. Confirm live weather/price data still works.

Done when: Uploading a leaf photo returns a real (not mocked) disease/pest prediction with a confidence score, shown live on the deployed app.

Chunk 7 — Real Price Alert (SMS/WhatsApp Simulation)

Owner: P3 | Timing: Day 12 (1 day)

You are making the price-alert feature actually send a message, using a free sandbox — no real telecom cost or setup needed.

What to build, step by step:

1. Sign up for a free Twilio Sandbox account (WhatsApp or SMS sandbox).
2. Build a simple UI where a farmer can set a price alert: pick a crop, a market, and a target price.
3. Build a backend table to store these thresholds.
4. Build a background job that checks current prices against stored thresholds (for the demo, this can be a button you click, or a job that runs every couple of minutes).
5. When a price crosses a threshold, send a real test message through the Twilio Sandbox to a test phone number.
6. Test the full loop end-to-end: set a threshold, force it to be crossed (or use test data), and confirm the message actually arrives.
7. Deploy on top of Chunk 6's live app. Confirm Modules A and B and the leaf classifier still work.

Done when: Setting a price threshold and having the price cross it sends a real (sandbox) SMS or WhatsApp message, without breaking anything built earlier.

Chunk 8 — Testing, Monitoring & Demo Readiness

Owner: P4 | Timing: Day 13–14 (2 days)

You are making the whole app solid and making sure the team can demo it confidently in front of judges, twice in a row, without anything breaking.

What to build, step by step:

1. Day 13 — Write automated tests (pytest) covering the main logic: the advisory ranking engine, the spoilage model, and the decision engine. Cover the normal cases and a few edge cases.
2. Day 13 — Set up GitHub Actions so these tests run automatically on every push, and block merging if a test fails.

3. Day 13 — Set up Sentry (or a similar free tool) on both frontend and backend so errors get logged automatically during the demo.
4. Day 13 — Do a final UI polish pass: fix any rough edges, re-check the layout on a phone screen.
5. Day 14 — Write and rehearse a demo script: the exact order you'll click through, and the key talking points for problem, approach, and market fit.
6. Day 14 — Prepare for judge Q&A: write down 5–10 questions you think judges might ask, and agree on answers as a team.
7. Day 14 — Run the full demo twice back-to-back to confirm nothing breaks the second time (stale data, leftover test alerts, etc.).

Done when: Tests run automatically and pass on every push, errors are being tracked, and the team can run the full demo twice in a row without any manual fixes in between.

4. Phase 2 — Pilot (Month 1–3 After the Hackathon)

This phase only happens if you're moving forward with a real FPO (Farmer Producer Organisation) or KVK (Krishi Vigyan Kendra) partner after the hackathon. Goal: turn the hackathon MVP into something a real partner organisation can actually use with real farmers. Each chunk here takes about 2–3 weeks instead of days, since it's real infrastructure work.

Chunk 9 — Move to Real Cloud Infrastructure

Owner: P1 | Timing: ≈2–3 weeks

You are moving the app off free hackathon-tier hosting onto proper, managed cloud infrastructure that can handle real users.

What to build, step by step:

1. Containerise the app with Docker so it runs identically everywhere.
2. Move hosting to a managed cloud provider (AWS or GCP).
3. Set up a managed PostgreSQL database (with PostGIS for location data) to replace SQLite.
4. Add a Redis cache for frequently-requested data like weather and prices.
5. Set up a proper staging environment separate from the live hackathon demo, so you can test changes safely.
6. Confirm the old hackathon demo link still works as a fallback while this migration happens.

Done when: The app runs identically on the new cloud infrastructure in a staging environment, and the old demo link still works as a backup.

Chunk 10 — Field-Agent Access & Local Language

Owner: P2 | Timing: ≈2–3 weeks

You are adding the ability for a field agent (who works with many farmers) to log in and manage records for multiple farmers, in their own language.

What to build, step by step:

1. Build a login/authentication system for field agents.
2. Build a way for one field agent to create and manage records for many farmers (batch entry).
3. Translate the advisory and post-harvest UI text into Hindi and Gujarati, and add a language toggle.
4. Wire the language toggle end-to-end so switching it actually changes all visible text.
5. Test the whole flow as a field agent: log in, add several farmers, switch languages, and confirm everything works on the staging environment.

Done when: A field agent can log in, create records for multiple farmers, and switch the interface language — all working on the staging environment.

Chunk 11 — Real Feedback Loop & Real Messaging

Owner: P3 | Timing: ≈2–3 weeks

You are replacing the sandbox alert system with real messaging, and starting to collect real feedback from farmers on whether the advisories are actually useful.

What to build, step by step:

1. Add a thumbs-up / thumbs-down button on every advisory shown to a farmer, and store this feedback against that advisory.
2. Apply for and set up a verified WhatsApp Business API sender, replacing the Twilio sandbox from the hackathon.
3. Set up monitoring dashboards (Prometheus/Grafana) to track live traffic and error rates.
4. Test the full real-messaging flow with a small group of test numbers before rolling out to real farmers.

Done when: Feedback is being stored against each advisory, real WhatsApp alerts are being sent (not sandbox), and dashboards show live traffic and error rates.

Chunk 12 — Pilot Hardening & Handover to Partner

Owner: P4 | Timing: ≈2–3 weeks

You are making the app reliable enough for real, everyday use in the field, and handing it over to the pilot partner organisation.

What to build, step by step:

1. Add offline caching (PWA) so the app still shows the last-known data if a farmer loses signal in the field.
2. Set up proper secrets management (API keys, passwords) instead of hardcoded values.
3. Add PII protection: encrypt sensitive farmer data, and store location as a coarse pin rather than an exact address.
4. Write partner-facing documentation: how to use the app, who to contact for support, and what data is collected.
5. Deploy to production and confirm the pilot partner (FPO/KVK) can access and use it.

Done when: The app is live in production for the pilot partner, works reasonably well offline, and documentation has been handed over.

5. Phase 3 — Scale (Month 3–12 After the Hackathon)

This is the long-term phase: growing from one pilot partner to many, and from a rule-based system to one that learns from real data. Each chunk here takes about 4–6 weeks.

Chunk 13 — Split Into Separate Services

Owner: P1 | Timing: ≈4–6 weeks

You are breaking the single app into smaller, independent services so the system can grow without one part slowing down the others.

What to build, step by step:

1. Split the app into three separate services: Advisory, Market Intelligence, and Alerts.
2. Put an API gateway in front of all three so the outside world still sees one consistent app.
3. Set up Terraform so cloud environments can be created and rebuilt consistently, not by hand.
4. Test thoroughly to confirm nothing that worked in the pilot stopped working after the split.

Done when: All three services run independently behind the gateway, with no loss of functionality that existed during the pilot.

Chunk 14 — Mobile App

Owner: P2 | Timing: ≈4–6 weeks

You are building a proper mobile app so farmers and field agents don't need to rely on a mobile browser.

What to build, step by step:

1. Build a React Native (Expo) app that reuses the same visual design as the web app.
2. Add offline-first syncing, so data entered offline uploads automatically once signal returns.
3. Test that every feature in the web app also works in the mobile app.

Done when: The mobile app can be installed, works offline-first, and matches the web app's features when tested against the live services.

Chunk 15 — Smarter Models From Real Field Data

Owner: P3 | Timing: ≈4–6 weeks

You are starting to replace the rule-based logic with models trained on real data collected from the pilot, while keeping a safe fallback.

What to build, step by step:

1. Set up a feature store to organise the real farmer/field data collected during the pilot.
2. Set up a scheduled retraining pipeline (Airflow or Prefect) so models can be refreshed as new data comes in.
3. Train the first real model on field data to replace part of the rule engine, and track it with MLflow.
4. Put the new model behind a feature flag so it can be turned on for a small % of users first, with an easy rollback to the rule engine if something looks wrong.

Done when: A trained model is live behind a feature flag, and rolling back to the rule engine has been tested and confirmed to work.

Chunk 16 — Support Many Partners at Once

Owner: P4 | **Timing:** ≈4–6 weeks

You are making the system ready to support multiple FPOs/KVKs at the same time, not just the original pilot partner.

What to build, step by step:

1. Build a tenant-onboarding flow so a new FPO/KVK can be added without writing new code.
2. Add a voice/IVR channel for farmers who can't use a smartphone app easily.
3. Set up Kubernetes autoscaling so the system can handle more users without manual intervention.
4. Set up centralised logging across all services and add billing/subscription tracking if the business model needs it.
5. Onboard a second FPO/KVK as a real test of the new flow.

Done when: A second FPO/KVK partner can be onboarded end-to-end with no code changes, and the system automatically scales under increased load.

6. Quick Reference — Who Builds What, When

Phase	P1	P2	P3	P4
Phase 0 — Prototype (Week 1, Day 1–7)	Foundation + Rules (Day 1)	Crop Advisory Module (Day 2–3)	Post-Harvest Module (Day 4–5)	Integration + Submission (Day 6–7)
Phase 1 — Hackathon MVP (Week 2, Day 8–14)	Live Weather + Prices (Day 8–9)	Leaf Classifier (Day 10–11)	Real Price Alert (Day 12)	Testing + Demo Ready (Day 13–14)
Phase 2 — Pilot (Month 1–3)	Cloud Infrastructure	Field-Agent + Language	Feedback + Real Messaging	Hardening + Handover
Phase 3 — Scale (Month 3–12)	Split Into Services	Mobile App	Smarter Models	Multi-Partner Support

Notice the pattern: it's always P1 → P2 → P3 → P4, in every phase. Nobody is stuck doing only frontend or only backend — everyone gets a turn building a complete feature from scratch.

7. The Simple Rules, One More Time

- Only one person actively builds at a time. Never two people editing the same feature together.
- Every handoff leaves the app fully working — deployed and live, not half-built.
- Before you start your chunk, make sure the previous person's chunk is merged, deployed, and their handoff note is posted.
- If you get stuck, don't sit on it — timebox the problem (1–2 hours in Week 1–2, a bit more later) and then ask the next person in line to help unblock you.
- If you're running out of time on your chunk, cut scope down to the "Done when" line — don't push the whole schedule back.